

Factory Planning 2

Problem

- Find the best production plan **and** the best repair cycle (maximize profit)

Instead of enforcing pre-decided months for maintenance we now include this planning in the production plan. The model is almost the same.

Extra Decision variables

- $q_{m,t} \in \{0, 1, 2, \dots\}$: number of machines of type m to repair in month t .

New parameter

- Number of machines that need to be repaired of each type: $RepairsNeeded_m$
Note that this can be extracted from the previous data by summing how many forced repairs there were over the months:

$$RepairsNeeded_m = \sum_t MachineRepairs_{m,t} \quad \forall m$$

Alternatively, it can be taken from the assignment text, giving the same result:

$$RepairsNeeded = (2, 2, 3, 1, 1)$$

In the model from previous we only need to add one constraint and modify another, hence we only specify the changes:

Model

Constraints (changed or added):

- Machine capacity limit is the same, except that parameter $MachineRepairs_{m,t}$ is replaced by variable $q_{m,t}$:

$$\sum_p ProcessTime_{m,p} \cdot x_{p,t} \leq$$

$$(NumMach_m - q_{m,t}) \cdot WorkingHoursPrMonth \quad \forall m, t$$

- Ensure that the right number of machines are actually repaired:

$$\sum_t q_{m,t} = RepairsNeeded_m \quad \forall m$$

The full model in Julia/JuMP, available with the name

FactoryPlanning2.jl

from the book web-site, is given below:

```

*****
# Factory Planning Assignment, "Mathematical Programming Modelling" (42112)
*****

# Intro definitions
using JuMP
using HiGHS
*****

# PARAMETERS
Machines = ["Grinding", "Vertical drilling",
            "Horizontal drilling", "Boring", "Planing"]
M=length(Machines)
Products = ["PROD1", "PROD2", "PROD3", "PROD4",
            "PROD5", "PROD6", "PROD7"]
P=length(Products)
Time = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
T = length(Time)

# data
Profit = [10 6 8 4 11 9 3]
ProcessTime = # machine, product
    [0.5  0.7  0.0  0.0  0.3  0.2  0.5; # Grinding
     0.1  0.2  0.0  0.3  0.0  0.6  0.0; # Vertical drilling
     0.2  0.0  0.8  0.0  0.0  0.0  0.6; # Horizontal drilling
     0.05 0.03 0.0  0.07 0.1  0.0  0.08; # Boring
     0.0  0.0  0.01 0.0  0.05 0.0  0.05 # Planing
    ]
MachineRepairs = # time, machine
    #G  VD  HD  B  P
    [1.0  0.0  0.0  0.0  0.0  ; # Jan

```

```

0.0 0.0 2.0 0.0 0.0 ; # Feb
0.0 0.0 0.0 1.0 0.0 ; # Mar
0.0 1.0 0.0 0.0 0.0 ; # Apr
1.0 1.0 0.0 0.0 0.0 ; # May
0.0 0.0 1.0 0.0 1.0 # Jun
]

RepairsNeeded=sum(MachineRepairs, dims=1) #RepairsNeeded=[2,2,3,1,1]

NumMach = [4 2 3 1 1]

StorageCap = 100
StorageCost = 0.5
EndStock = 50
WorkingHoursPrMonth = 24*8*2 # no working days, 2 shifts of 8 hours

Demand = # product, month
#Jan #Feb #Mar #Apr #May #Jun
[500 600 300 200 0 500; #PROD1
1000 500 600 300 100 500; #PROD2
300 200 0 400 500 100; #PROD3
300 0 0 500 100 300; #PROD4
800 400 500 200 1000 1100; #PROD5
200 300 400 0 300 500; #PROD6
100 150 100 100 0 60; #PROD7
]

*****

#*****
# Model
factoryplanning2 = Model(HiGHS.Optimizer)

# production of product p in month t
@variable(factoryplanning2, x[1:P,1:T] >= 0)

# sale of product p in month t
@variable(factoryplanning2, y[1:P,1:T] >= 0)

# storage of product p in THE END of month t
@variable(factoryplanning2, s[1:P,1:T] >= 0)

# number of machines of type m down for repair in month t
@variable(factoryplanning2, 0 <= q[1:M,1:T] <= 4, Int)

# maximize profit

```

```

@objective(factoryplanning2, Max,
            sum(Profit[p]*y[p,t] for p=1:P, t=1:T) -
            sum(StorageCost*s[p,t] for p=1:P, t=1:T))

# machine capacity limit constraint
@constraint(factoryplanning2, [m=1:M,t=1:T],
            sum(ProcessingTime[m,p]*x[p,t] for p=1:P) <=
            (NumMach[m] - q[m,t])*WorkingHoursPrMonth)

# storage balance constraint (ultimo storage)
@constraint(factoryplanning2, [p=1:P, t=1:T],
            s[p,t] == x[p,t] - y[p,t] + (t>1 ? s[p,t-1] : 0) )

# end storage
@constraint(factoryplanning2, [p=1:P],
            s[p,6] == EndStock)

# max storage
@constraint(factoryplanning2, [p=1:P, t=1:T],
            s[p,t] <= StorageCap)

# demand limit
@constraint(factoryplanning2, [p=1:P, t=1:T],
            y[p,t] <= Demand[p,t])

# force maintenance
@constraint(factoryplanning2, [m=1:M],
            sum(q[m,t] for t=1:T) == RepairsNeeded[m])

print(factoryplanning2)
#####

#####
# solve
optimize!(factoryplanning2)
println("Termination status: $(termination_status(factoryplanning2))")
#####

#####
# Report results
println("-----");
if termination_status(factoryplanning2) == MOI.OPTIMAL
    println("RESULTS:\n")
    println("objective = $(objective_value(factoryplanning2))")

```

```

else
    println(" No solution")
end
println("-----");
#####

#####
println("Successfull end of $(PROGRAM_FILE)")
#####

```

Comments to the Julia/JuMP program:

- Notice: The assignment is basically solved by replacing the maintenance matrix from **FactoryPlanning** with the new integer variable and adding a constraint that forces the variable to take the necessary number of non-zero values.
- In the code we set an upper bound on the new q -variable of 4. This is not necessary but can help the solver be faster. In an optimal solution there will never be more repairs scheduled than necessary, and we have no more than 4 of the same machine type so at the absolute most 4 could be off in the same month. You could strengthen the upper bound further by setting the upper bound to $RepairsNeeded_m$.